

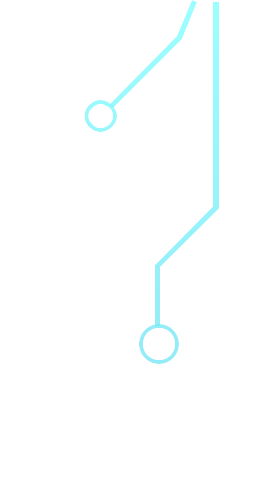
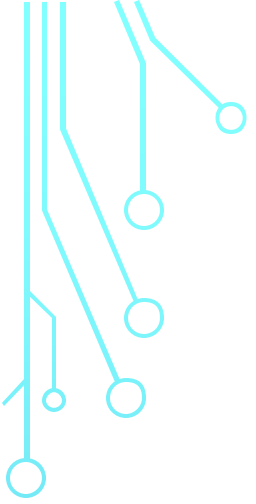


Intro to C

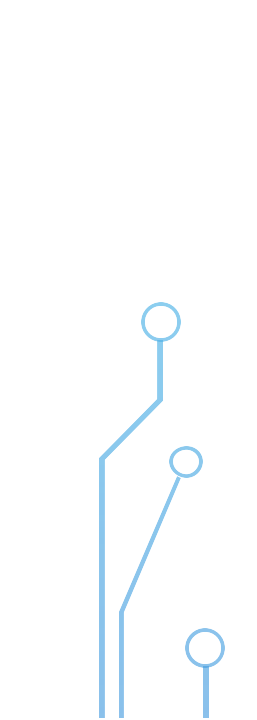
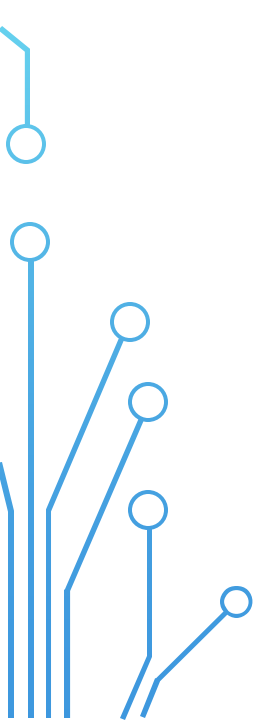
COMP211 Connor McMahon

Announcements

- Reminder: learncli guided reading questions due on Wednesday.
- Guided reading questions grading
 - Will be averaged and count for two homework assignments
 - Immediate feedback is not given for the guided reading questions
- Next guided reading assignment is due next Wednesday (Chapter 1)
 - Today, we'll be covering sections 1.1-1.3
 - You'll read section 1.4 on your own
 - As you are reading the textbook, I encourage you to complete the book's provided exercises at the end of each section.
- Slides with solutions to the active learning problems are posted at the end of the day.



Binary, Hex, and Decimal Reading Recap



Reading Review: Binary and Hexadecimal

Decimal	Binary	Hexadecimal
0	0b 0000	0x0
1	0b 0001	0x1
2	0b 0010	0x2
3	0b 0011	0x3
4	0b 0100	0x4
5	0b 0101	0x5
6	0b 0110	0x6
7	0b 0111	0x7
8	0b 1000	0x8
9	0b 1001	0x9
10	0b 1010	0xA
11	0b 1011	0xB
12	0b 1100	0xC
13	0b 1101	0xD
14	0b 1110	0xE
15	0b 1111	0xF

You should know the
hex and binary
representation of the
numbers 0 through 15
by heart <3

Binary

- There are two ways to indicate that a number is in binary
 - Prepend 0b
 - Ex: 0b1001
 - Subscript 2
 - Ex: 1001₂
- Group the digits of a binary number in sets of four to make it easier to read
 - Normally, spaces are used when handwritten
 - Ex: 0b1010 1111 0101
 - And underscores are used for typed numbers
 - Ex: 0b1010_1111_0101
 - If you cannot evenly break up the digits into groups of four, start making the groups from the right
 - Ex: 0b10_1010_0011

Hex

- There are two ways to indicate that a number is in hex
 - Prepend 0x
 - Ex: 0xAB
 - Subscript 16
 - Ex: AB₁₆
- Group the digits of a hex number in sets of four to make it easier to read
 - Normally, spaces are used when handwritten
 - Ex: 0xABCD 1234
 - And underscores are used for typed numbers
 - Ex: 0bABCD_1234
 - If you cannot evenly break up the digits into groups of four, start making the groups from the right
 - Ex: 0bAB_CDEF

Binary Bit Positions

- We'll often want to reference individual digits of a binary number
- To do this, we need to give a name to each bit
- The bits in a binary number are labeled from $n-1$ to 0, where n represents the total number of bits in the number.

1 0 1 1₂

Bit 3 Bit 2 Bit 1 Bit 0

Hex Positions

- A hexadecimal digit is called a **hexit**.
- Each hexit is made up of 4 bits.
- The number below contains four hexits, which is a total of 16 bits.

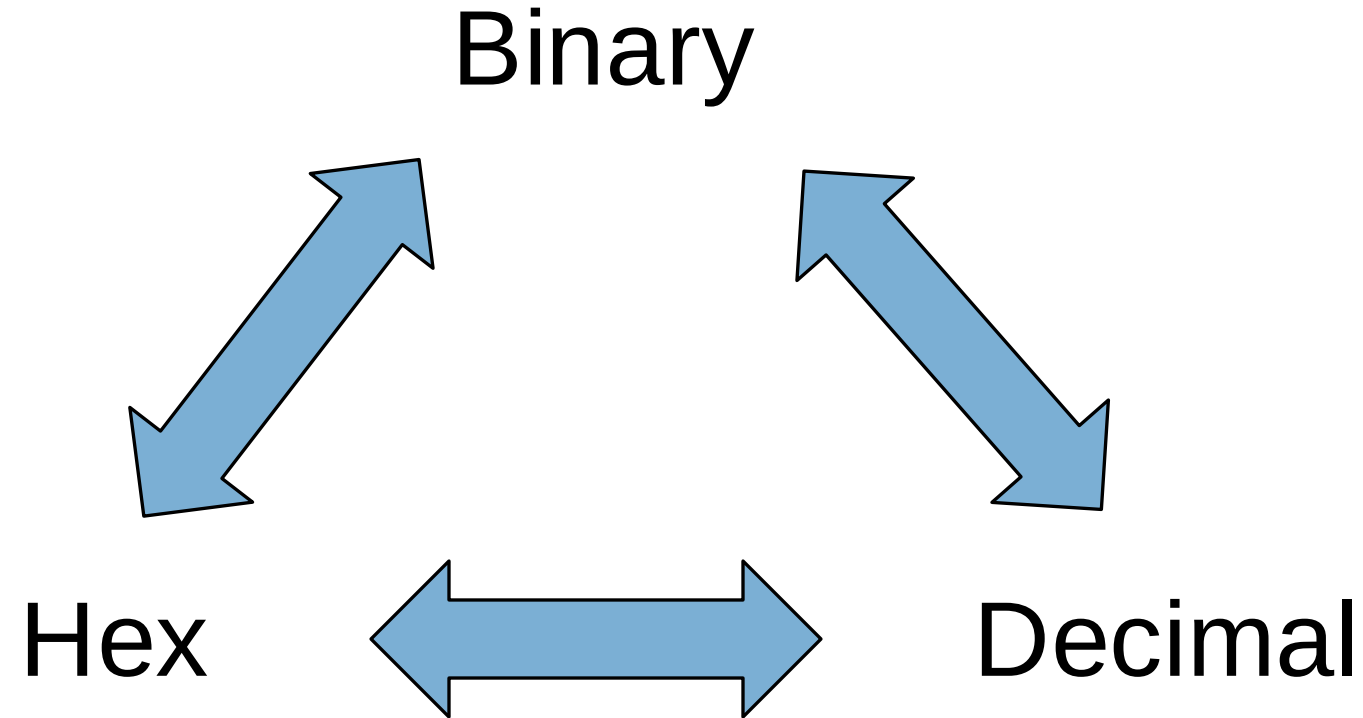
7 D B 4₁₆

Hexit 3 Hexit 1

 Hexit 2 Hexit 0

Reading Review

- Make sure you are comfortable converting between binary, hex, and decimal.



Compiling and Assembling

Machine Code

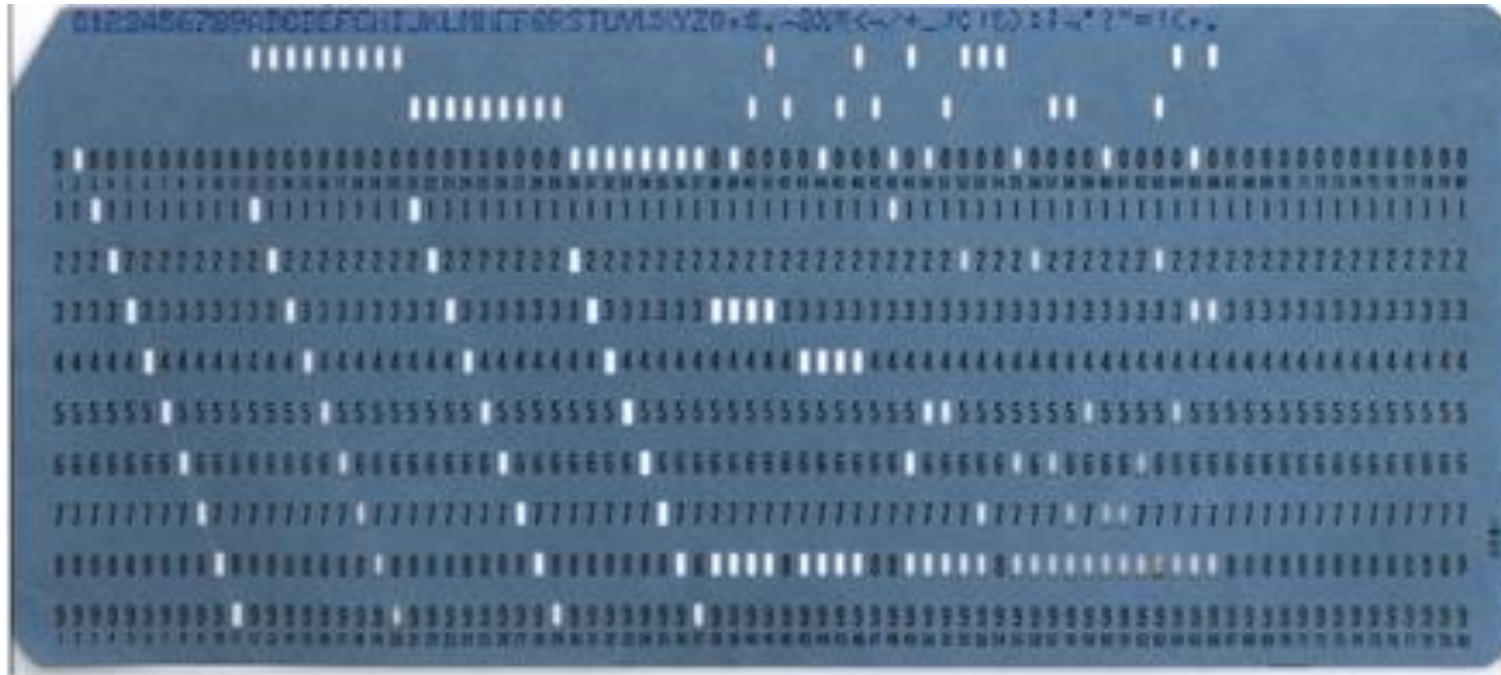
- Computers cannot directly understand the programs that we write (Python, Java, etc). Instead, they understand something called machine code.
- **Machine code** consists of binary instructions (strings of 1s and 0s) that tell the computer what operations to perform.

Machine code

```
00100111101111011111111111111000
10101111101111100000000000000100
00000011101000001111000000100101
10101111110001000000000000001000
10101111110000100000000000001000
00000000000000000000000000000000
00000000010000100000000000011000
00000000000000000000100000010010
00000011110000001110100000100101
10001111101111100000000000000100
00100111101111010000000000001000
0000001111100000000000000001000
00000000000000000000000000000000
```

A Brief History of Programming

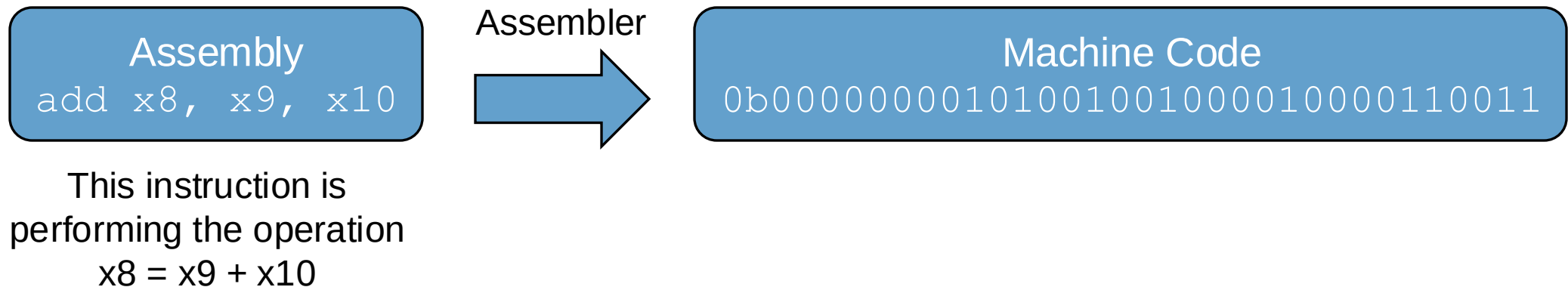
- Before the 1950s, programming was done entirely in **machine code**
- These instructions were entered manually using switches, punch cards, or paper tape.



IBM 80-Column Punch Card

A Brief History of Programming

- In the early 1950s, **assembly language** was introduced as a more human-readable way to write programs.
- Machines cannot read assembly language programs (they can only read machine code)!
- **Assemblers** (programs that translate assembly code into machine code) were developed to automate the conversion process.
- There are lots of different assembly languages. You'll learn more about them in COMP311!



A Brief History of Programming

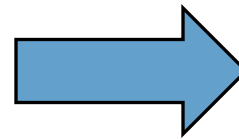
- With the invention of assembly language, programmers could write and debug code much more efficiently!
- Assembly language is known as a **low-level language**

Assembly Language
(Low-level language)

```
1 square(int):  
2     addiu    $sp,$sp,-8  
3     sw       $fp,4($sp)  
4     move     $fp,$sp  
5     sw       $4,8($fp)  
6     lw       $2,8($fp)  
7     nop  
8     mult     $2,$2  
9     mflo     $2  
10    move     $sp,$fp  
11    lw       $fp,4($sp)  
12    addiu    $sp,$sp,8  
13    jr       $31  
14    nop
```

Human readable

Assembler



Machine code

```
001001111011110111111111111111000  
10101111101111100000000000000100  
00000011101000001111000000100101  
1010111111000100000000000001000  
10101111110000100000000000001000  
00000000000000000000000000000000  
000000000100001000000000000011000  
000000000000000000001000000010010  
00000011110000001110100000100101  
10001111101111100000000000000100  
00100111101111010000000000001000  
00000011111000000000000000001000  
00000000000000000000000000000000
```

Machine-readable

A Brief History of Programming

- In the late 1950s, **high-level languages** began abstracting away the details of hardware, focusing on logic and algorithms.
- Key Early Languages:
 - 1957: FORTRAN (FORMula TRANslation):
 - First widely adopted high-level language.
 - Developed by IBM for scientific and engineering calculations.
 - 1958: LISP (LIST Processor):
 - Pioneered functional programming and symbolic processing.
 - 1959: COBOL (COMmon Business-Oriented Language):
 - Designed for business applications.
 - Emphasized readability and portability.
 - Became a standard for enterprise and financial systems.

High-Level Languages

- Most programming languages you've heard of are **high-level programming languages** (Python, Java, C, C++).
- C is a **compiled high-level programming language**.
- To execute a compiled high-level programming language, the program is first translated into assembly language by a **compiler**, then an **assembler** translates the assembly language into machine code.
- **Interpreted languages** like Python are executed differently. We'll talk more about interpreted languages later.

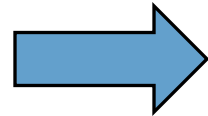
Assembling and Compiling

C
(High-level language)

```
1 int square(int num) {  
2     return num * num;  
3 }
```

Easy for humans to
read/write

Compiler



Assembly Language
(Low-level language)

```
1 square(int):  
2     addiu    $sp,$sp,-8  
3     sw       $fp,4($sp)  
4     move     $fp,$sp  
5     sw       $4,8($fp)  
6     lw       $2,8($fp)  
7     nop  
8     mult     $2,$2  
9     mflo     $2  
10    move     $sp,$fp  
11    lw       $fp,4($sp)  
12    addiu    $sp,$sp,8  
13    jr       $31  
14    nop
```

Human readable

Assembler



Machine code

```
0010011110111101111111111111111000  
101011111011111000000000000000100  
00000011101000001111000000100101  
10101111110001000000000000001000  
10101111110000100000000000001000  
00000000000000000000000000000000  
00000000010000100000000000011000  
00000000000000000001000000010010  
00000011110000001110100000100101  
10001111101111100000000000000100  
00100111101111101000000000001000  
0000001111100000000000000001000  
00000000000000000000000000000000
```

Machine-readable

Key Terms

- Machine Code
- Assembly Language
- Low-level language
- High-level language
- Compiler
- Assembler
- Compiled language
- FORTRAN, LISP, COBOL

Poll Everywhere

1. What is machine code?

- A. A high-level programming language like Python
- B. The binary instructions executed directly by a CPU
- C. A human-readable form of computer instructions
- D. A type of compiler used for low-level programming

2. Assembly language is best described as:

- A. A human-readable representation of machine code
- B. A type of high-level language
- C. A translator for converting machine code to high-level language
- D. A language executed directly by the CPU

3. High-level languages are designed to:

- A. Be executed directly by hardware.
- B. Simplify programming by abstracting hardware details.
- C. Provide binary-level control over a CPU.
- D. Translate assembly code into machine code.

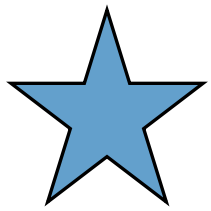
4. What is the primary role of a compiler?

- A. Translate assembly code into machine code.
- B. Execute high-level code line-by-line.
- C. Convert high-level language code into assembly.
- D. Translate machine code into high-level code.

5. What is the relationship between assembly language and machine code?

- A. Assembly language is compiled into machine code.
- B. Assembly language is interpreted into high-level code.
- C. Assembly language is translated into machine code using an assembler.
- D. Assembly language is another term for machine code.

Poll Everywhere



1. What is machine code?

- A. A high-level programming language like Python
- B. The binary instructions executed directly by a CPU**
- C. A human-readable form of computer instructions
- D. A type of compiler used for low-level programming

2. Assembly language is best described as:

- A. A human-readable representation of machine code**
- B. A type of high-level language
- C. A translator for converting machine code to high-level language
- D. A language executed directly by the CPU

3. High-level languages are designed to:

- A. Be executed directly by hardware.
- B. Simplify programming by abstracting hardware details.**
- C. Provide binary-level control over a CPU.
- D. Translate assembly code into machine code.

4. What is the primary role of a compiler?

- A. Translate assembly code into machine code.
- B. Execute high-level code line-by-line.
- C. Convert high-level language code into assembly.**
- D. Translate machine code into high-level code.

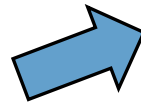
5. What is the relationship between assembly language and machine code?

- A. Assembly language is compiled into machine code.
- B. Assembly language is interpreted into high-level code.
- C. Assembly language is translated into machine code using an assembler.**
- D. Assembly language is another term for machine code.

C Introduction

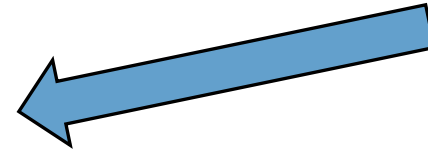
Your First C Program

`main` returns an integer value that is used to indicate the exit status of the program. A successful execution returns 0. An abnormal execution returns a non-zero value.



```
#include <stdio.h>
```

```
int main() {  
    printf("Hello, world!\n");  
    return 0;  
}
```



All C programs begin executing at the main function.

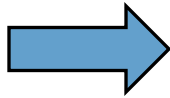
Your First C Program

To use `printf`, you must explicitly include `stdio.h`.



```
#include <stdio.h>
```

`printf` is a standard library function that prints output.



```
int main() {  
    printf("Hello, world!\n");  
    return 0;  
}
```

Standard Library

- The **C Standard Library** contains a collection pre-written functions that handle common tasks like input/output, string manipulation, mathematical operations, and memory management.
- Java also has a standard library which offers a much broader range of functionalities.

C	Java
• Library functions are grouped together in header files. • C does not automatically include any standard library functions.	• Library methods are organized into classes within packages. • Java automatically includes classes from the java.lang package. These classes are always available, so you don't need to explicitly import them.

Header Files

- A header file contains the declarations of the functions, macros, and types you can use.
 - Function Declaration: the function's name, return type, and parameters
 - Function Definition: the body of the function

Header File	Purpose	Examples of Functions
<code><stdio.h></code>	Input/output functions	<code>printf, getchar, putchar, fopen</code>
<code><stdlib.h></code>	General utilities	<code>malloc, free, exit, atoi, rand</code>
<code><string.h></code>	String manipulation	<code>strlen, strcpy, strcat, strcmp</code>
<code><math.h></code>	Mathematical operations	<code>sqrt, pow, sin, cos, tan, exp</code>
<code><time.h></code>	Date and time utilities	<code>time, clock, localtime, gmtime</code>

Compiling, Assembling, and Running your First C Program

1. Write your program.
2. Save the file with a .c extension.
 - For example, `hello.c`
3. Open a terminal and navigate to the directory where your program is located.
4. Compile and assemble the program
 - `gcc hello.c`
5. Run the program
 - `./a.out`

`hello.c`

```
#include <stdio.h>

int main() {
    printf("Hello, world!\n");
    return 0;
}
```

Compiling and Assembling your Program

- `gcc hello.c` converts your human-readable C code into a machine-readable **executable file**.
- `gcc hello.c`
 - `gcc`: invokes the GNU C Compiler
 - `hello.c` is the source file you want to compile
 - By default, this produces an executable file named `a.out`.
 - If you want to name your executable file something else, let's say you want to call it `hello`, you would use the following command
 - `gcc hello.c -o hello`

GNU

- Pronounced as one syllable with a hard *g*, like “grew” but with the letter *n* instead of *r*.
- “GNU” is a recursive acronym for “GNU's Not Unix!”
- Developed in response to AT&T charging for the Unix operating system (when it was previously free)
- The free software concept originated with GNU
- GNU is now an extensive collection of free software that has been used to create the family of operating systems known as GNU/Linux

Running your Program

- `./a.out`
- `.` refers to the **current directory**, which is the directory (folder) you're in when you run the command.
- When you run the command `./a.out`, you're instructing the system to locate the file named `a.out` in the current directory and run it.
- If you name your executable something else (e.g., `hello`) using the `-o` option in `gcc` (e.g., `gcc hello.c -o hello`), you would run your program with:
 - `./hello`

Poll Everywhere

- Which of the following set of commands can you use to compile, assemble, and execute the program in a file named `my_program.c`

A

```
gcc my_program.c  
./my_program.c
```

B

```
gcc my_program.c -o my_program  
./a.out
```

C

```
gcc my_program.c -o my_prog  
./my_prog
```

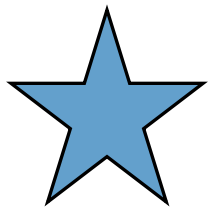
D

```
gcc my_program.c  
./a.out
```

E

```
gcc my_program.c -o my_program  
./my_program.c
```

Poll Everywhere



- Which of the following set of commands can you use to compile, assemble, and execute the program in a file named `my_program.c`

A

```
gcc my_program.c  
./my_program.c
```

B

```
gcc my_program.c -o my_program  
./a.out
```

C

```
gcc my_program.c -o my_prog  
./my_prog
```

D

```
gcc my_program.c  
./a.out
```

E

```
gcc my_program.c -o my_program  
./my_program.c
```

Key Terms

- Standard Library
- main function
- gcc
- GNU
- Executable

Fahrenheit to Celsius

A Java program to print a table of Fahrenheit temperatures and their Celsius equivalents.

Program Output.

```
public class TemperatureConverter {  
    public static void main(String[] args) {  
        float fahr, celsius;  
        int lower, upper, step;  
  
        lower = 0; // lower limit of temperature table  
        upper = 300; // upper limit  
        step = 20; // step size  
  
        fahr = lower;  
        while (fahr <= upper) {  
            celsius = (5.0f / 9.0f) * (fahr - 32.0f);  
            System.out.printf("%3.0f %6.1f%n", fahr, celsius);  
            fahr = fahr + step;  
        }  
    }  
}
```

0	-17.8
20	-6.7
40	4.4
60	15.6
80	26.7
100	37.8
120	48.9
140	60.0
160	71.1
180	82.2
200	93.3
220	104.4
240	115.6
260	126.7
280	137.8
300	148.9

Java vs. C

Java

```
public class TemperatureConverter {  
    public static void main(String[] args) {  
        float fahr, celsius;  
        int lower, upper, step;  
  
        lower = 0;  
        upper = 300;  
        step = 20;  
  
        fahr = lower;  
        while (fahr <= upper) {  
            celsius = (5.0f / 9.0f) * (fahr - 32.0f);  
            System.out.printf("%3.0f %6.1f\n", fahr,  
celsius);  
            fahr = fahr + step;  
        }  
    }  
}
```

C

```
#include <stdio.h>  
  
int main() {  
    float fahr, celsius;  
    int lower, upper, step;  
  
    lower = 0;  
    upper = 300;  
    step = 20;  
  
    fahr = lower;  
    while (fahr <= upper) {  
        celsius = (5.0 / 9.0) * (fahr - 32.0);  
        printf("%3.0f %6.1f\n", fahr, celsius);  
        fahr = fahr + step;  
    }  
}
```

Java vs. C

Java

```
public class TemperatureConverter {  
    public static void main(String[] args) {  
        float fahr, celsius;  
        int lower, upper, step;  
  
        lower = 0;  
        upper = 300;  
        step = 20;  
  
        fahr = lower;  
        while (fahr <= upper) {  
            celsius = (5.0f / 9.0f) * (fahr - 32.0f);  
            System.out.printf("%3.0f %6.1f\n", fahr,  
celsius);  
            fahr = fahr + step;  
        }  
    }  
}
```

C

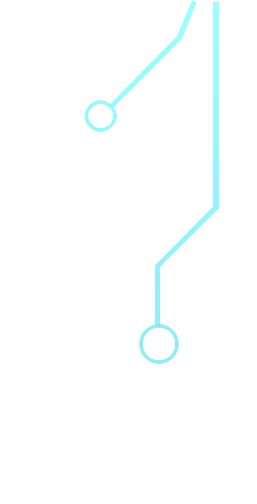
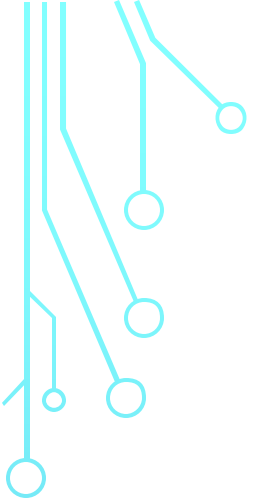
```
#include <stdio.h>  
  
int main() {  
    float fahr, celsius;  
    int lower, upper, step;  
  
    lower = 0;  
    upper = 300;  
    step = 20;  
  
    fahr = lower;  
    while (fahr <= upper) {  
        celsius = (5.0 / 9.0) * (fahr - 32.0);  
        printf("%3.0f %6.1f\n", fahr, celsius);  
        fahr = fahr + step;  
    }  
}
```

C-Style Languages

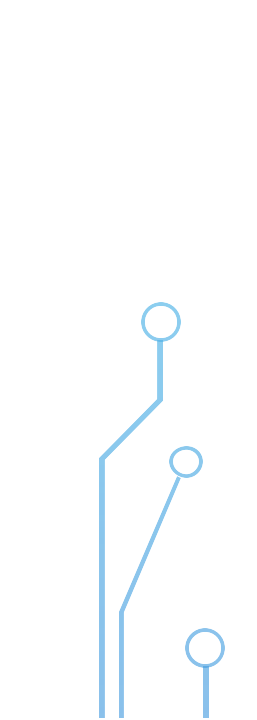
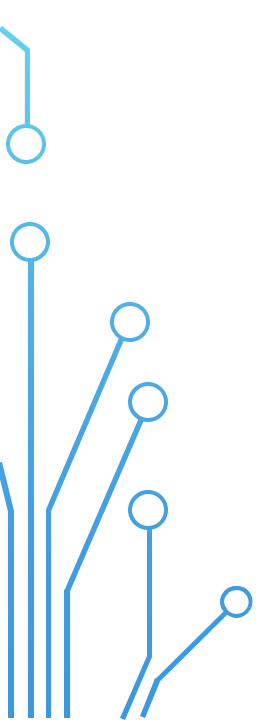
- Java and C are syntactically similar because Java is a member of the **C-style** language family.
- **C-style** languages are syntactically based on C Programming Language.
- There are 70 C-style languages
- Other C-style languages include Go, Rust, Swift, C++, and JavaScript

OOP vs. Procedural Programming Paradigms

- Java is an **object-oriented programming (OOP) language**, which organizes code around objects that encapsulate data and functions.
- C is a **procedural programming language**, which organizes code around procedures, also called functions.
 - There are no classes/objects in C!

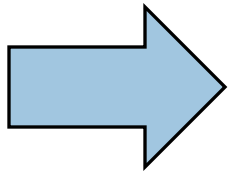


Stepping Through Programs in VS Code



Stepping Through Programs in VS Code

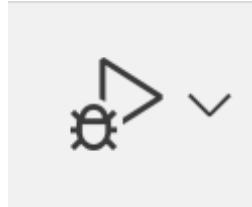
- Place a breakpoint in your program by clicking to the left of the line number where you would like to place the breakpoint. A red dot should appear.



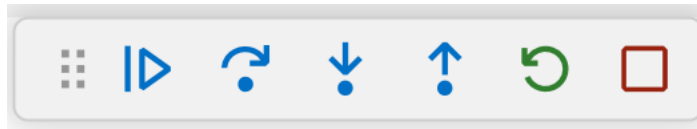
```
1  #include <stdio.h>
2
3  int main() {
4      float fahr, celsius;
5      int lower, upper, step;
6
7      lower = 0;    // lower limit of temperature table
8      upper = 300;  // upper limit
9      step = 20;    // step size
10
11     fahr = lower;
12     while (fahr <= upper) {
13         celsius = (5.0 / 9.0) * (fahr - 32.0);
14         printf("%3.0f %6.1f\n", fahr, celsius);
15         fahr = fahr + step;
16     }
17 }
```

Stepping Through Programs in VS Code

- In the upper right-hand corner, click the debug button.



- This panel should appear. You'll use this to control stepping through the program.



Stepping Through Programs in VS Code

- Notice on the left panel that you can see the value of each variable. If you placed your breakpoint on line 7, you'll observe that all variables contain "garbage" values, as they have not yet been initialized. Note that your values may differ from those shown in the example below.

✓ VARIABLES

✓ Locals

fahr = 9.18340949e-41

celsius = 0

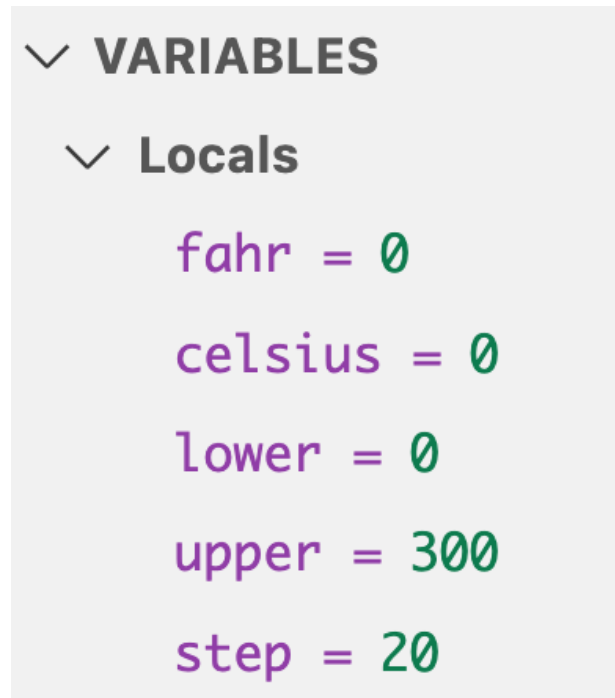
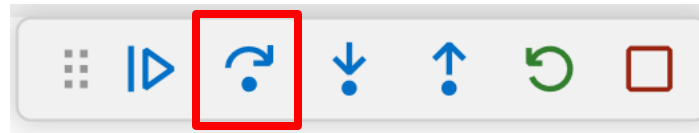
lower = -3800

upper = 65535

step = 16

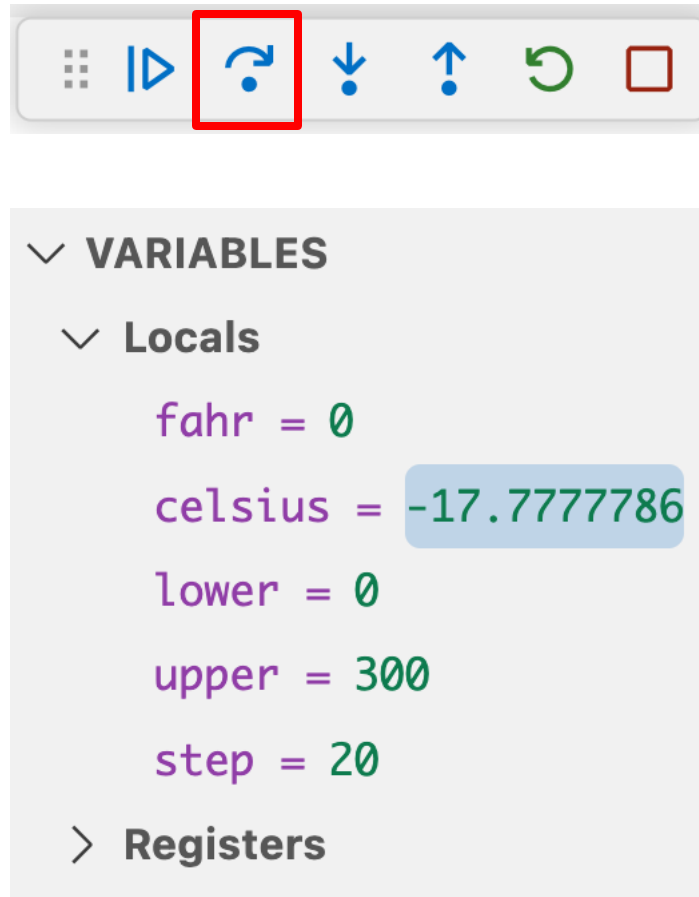
Stepping Through Programs in VS Code

- Click the **step-over** button until you are on line 12. At this point, you should see that all variables have been initialized with their assigned values.



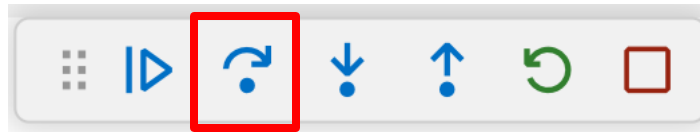
Stepping Through Programs in VS Code

- Click step-over until you are on line 14. You should now see that `celsius` has updated.



Stepping Through Programs in VS Code

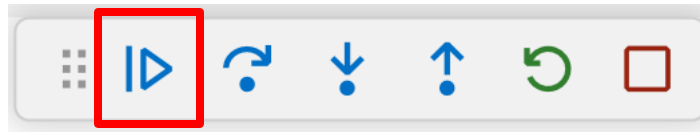
- Click step-over again. You should now see that the first row of the table has been printed to your terminal.



PROBLEMS	OUTPUT	DEBUG CONSOLE	<u>TERMINAL</u>	PORTS
			0 -17.8	

Stepping Through Programs in VS Code

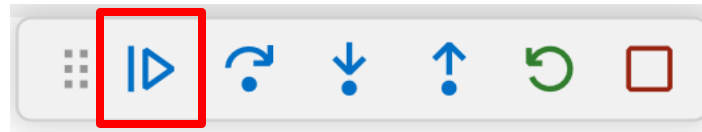
- Step again to get back to the start of the loop. Next, place a breakpoint on line 15 and click **continue**. The program will run until it reaches the breakpoint, at which point you should see that the next line of the table has been printed.



PROBLEMS	OUTPUT	DEBUG CONSOLE	<u>TERMINAL</u>	PORTS
0	-17.8			
20	-6.7			

Stepping Through Programs in VS Code

- Click continue again. Your program has executed another iteration of the loop, and you should see the next line of the table has been printed.



PROBLEMS	OUTPUT	DEBUG CONSOLE	<u>TERMINAL</u>	PORTS
0	-17.8			
20	-6.7			
40	4.4			

Stepping Through Programs in VS Code

- Clicking restart will restart the program and stop on the first breakpoint you set.



- Clicking stop will end the debugging session.



- We'll learn more about step into and step out later.

